

Technical Report

SE 801: Software Project Lab III

Supervised by:

Dr. Ahmedul Kabir
Associate Professor
Institute of Information Technology
University of Dhaka

Submitted by:

Name: Labib Muntasir
Roll: 1319

Submitted to:

SPL3 Program Committee

Submission date: 26 February, 2026

Letter of Transmittal

26th February, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Technical Report Submission.

Sir,

I am submitting the technical report for our Software Project Lab 3. This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement. Thank you for your consideration.

Sincerely,

Labib Muntasir

BSSE-1319

Institute of Information Technology

University of Dhaka

Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project. I would like to convey my tremendous gratitude to my supervisor, Dr. Ahmedul Kabir, Associate Professor, Institute of Information Technology, University of Dhaka, for providing me with guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

PlayDu is a mobile app created to help parents connect in a safe and supportive space. It feels familiar, like a social networking app, but it is built specifically for parents who want real and trustworthy connections. The goal is to create a digital community where parents can meet, share experiences, and support one another. Each parent creates a verified profile that includes their interests, parenting style, and family preferences. To make sure users are genuine, the app uses face detection and liveness checks through the camera. After that, identity verification is completed through a secure third-party service. This process helps build trust from the very beginning. PlayDu uses a smart matching system to suggest compatible parents. It looks at shared interests, parenting values, and location to recommend meaningful connections. The idea is similar to swipe-based apps like Tinder and Bumble, but redesigned for families and parenting communities. Once two parents match, they can chat securely, plan events, and even join local or interest-based groups. Beyond social interaction, PlayDu also offers educational resources and opportunities for cultural exchange. The project focuses strongly on privacy, security, and ease of use. It also explores how machine learning can improve personalization and trust in social platforms. At its heart, PlayDu aims to become a global digital village for parents, where technology helps build real connections, trust, and community.

1	Introduction	6
1.1	Purpose	6
1.2	Motivation	6
1.3	Scope	7
1.4	Purpose of Document	8
2	Project Description	9
2.1	Normal Requirements	9
2.2	Expected Requirements	10
2.3	Exciting Requirements	10
2.4	Usage Scenario	11
3	Scenario Based Modeling	12
3.1	Use Case Diagram	12
3.2	Use Case Level 0 (System Overview)	12
3.3	Use Case Level 1 (Detailed Modules)	13
4	Data Based Modeling	17
5	Class Based Modeling	24
5.1	Analysis Classes	24
5.2	Class Cards	25
5.3	CRC Diagram	34
6	Architectural Design	35
6.1	Architectural Context Diagram	35
6.2	Archetypes	35
6.3	Top Level Components	39
6.4	Deployment Diagram	39
7	Preliminary Test Plan	41
7.1	High Level Description of Testing goals	41
7.2	Test Cases	41
	Test Case 1: User Registration and Authentication	42
	Test Case 2: Parent Profile Creation	42
	Test Case 3: ID Verification Process	42
	Test Case 4: Liveness Check	43
	Test Case 5: Match Recommendation Generation	43
	Test Case 6: Swipe Actions (Like/Pass)	44
	Test Case 7: Mutual Match Creation	44
	Test Case 8: Secure In-App Messaging	44
	Test Case 9: User Reporting Mechanism	45
	Test Case 10: User Blocking Functionality	45

Test Case 11: Matching Algorithm Accuracy Validation	46
Test Case 12: Data Integrity and Validation	46
Test Case 13: Cross-Device Compatibility	46
Test Case 14: Accessibility Features	47
Test Case 15: Notification Delivery	47
Test Case 16: Session Management and Logout	48
8 Conclusion	49

1. Introduction

PlayDu is a social networking app designed specifically for parents who are looking for meaningful and trustworthy connections. Many parents today feel isolated, especially when they move to new places or struggle to find like-minded families nearby. PlayDu aims to rebuild that sense of community by creating a safe digital space where parents can connect, share experiences, and support one another.

It uses modern technology like machine learning to suggest compatible connections and strong verification methods to ensure users are genuine. The goal is simple: combine an easy-to-use mobile app with a secure backend system to create a supportive environment for parents both locally and globally. Through this platform, parents can build connections, exchange cultures, and create better learning and social opportunities for their children.

1.1 Purpose

The main purpose of PlayDu is to build a secure mobile platform where parents can form trusted communities. It focuses on social connection, cultural exchange, and educational support. By verifying user identities and using intelligent matching, the app helps parents find compatible peers. Key objectives:

- Create safe connections through AI-powered verification and smart matching.
- Provide communication tools, resource sharing features.
- Protect user data and give parents full control over their privacy to build long-term trust.

1.2 Motivation

In today's fast-moving world, many parents feel socially disconnected. Relocation, busy schedules, and the limitations of general social media platforms often make it difficult to form meaningful parenting communities. Most existing platforms are not designed specifically for families, and they rarely focus on strong identity verification.

It was created from the understanding that real support networks are essential for both emotional well-being and healthy child development. Inspired by swipe-based apps such as Tinder and Bumble, but redesigned for a family-friendly environment, PlayDu focuses on trust and compatibility.

- Address gaps in existing platforms by emphasizing trust through face detection.
- Utilize machine learning to deliver personalized recommendations, making connections more relevant and efficient.

- Explore how technology can enhance community building, as evidenced by the integration of hybrid recommendation systems (content-based and collaborative filtering).

Ultimately, PlayDu aims to create a global digital village where parents can form safe and meaningful relationships that positively influence their children's growth.

1.3 Scope

PlayDu is being developed as a cross-platform mobile application prototype suitable for a final semester project. The focus is on building core features while carefully selecting advanced functionalities that remain realistic within the time limit.

The project includes:

- User profile creation with identity verification such as face detection.
- Machine learning based matching for connecting parents.
- Secure communication features including in-app chat.
- Community tools like forums, groups.

The project does not include:

- Full production-level scalability and enterprise deployment features.
- Multilingual support beyond English.
- Payment gateways or advanced analytics integrations.
- Extensive testing across a wide range of physical mobile devices beyond emulation.

The development process follows privacy standards and uses open-source tools to ensure the project can be completed effectively within the semester timeline.

1.4 Purpose of Document

Its purpose is to:

- Document the project's journey from initial proposal to midterm prototype, including improvements based on feedback.
- Guide development by clearly defining requirements and usage scenarios.
- Make it easier for supervisors and peers to review the work.
- Provide a foundation for future phases such as integration and deployment while explaining technical decisions and challenges.

2. Project Description

PlayDu is a mobile application that establishes a secure and trustworthy digital community exclusively for parents. It enables verified parents to create detailed profiles, discover compatible connections through a smart matching system, and engage in meaningful conversations.

The app is designed to be simple, private, and personalized. It is built using React Native with Expo for the frontend, FastAPI for the backend, and accessible machine learning libraries.

2.1 Normal Requirements

These are the essential features required for a functional prototype. They form the foundation of the minimum viable product.

Functional Requirements

- User Profile Creation:
 - Parenting style such as attachment parenting, Montessori, or free-range
 - Personal interests such as hiking, book clubs, or cooking
 - Life stage tags such as new parent, single parent, homeschooling, or special needs

- User Login: Parents log in using Google account. Google OAuth 2.0 handles it.
- Basic Safety Controls: Parents can control profile visibility, report or block users, and easily access safety guidelines.

Non-Functional Requirements

- Usability: The app should have an intuitive and responsive interface built with React Native and Expo.
- Security and Privacy: JWT is used for authentication. Sensitive data is encrypted, and child information remains private by default.
- Performance: Profile loading, matching suggestions, and chat responses should occur within 2 to 4 seconds on mid-range devices.
- Reliability: The system should handle common errors gracefully and support small-scale testing with around 50 to 100 users.

2.2 Expected Requirements

These features enhance the core system once the basic prototype is stable.

Functional Requirements

- Parent Matching: A swipe-based interface allows parents to like or pass profiles. A match occurs when interest is mutual.
- Secure In-App Communication: Matched parents can send text messages within the app without sharing personal contact information.
- In-App Support: Users can submit queries or report issues through a simple support system.

Non-Functional Requirements

- Maintainability: The project should use structured code, GitHub version control, and proper documentation.
- Accessibility: Basic support for larger text sizes and high-contrast display modes.
- Scalability: Support future cloud deployment.

2.3 Exciting Requirements

These are stretch goals that add innovation but may not be fully implemented due to time constraints.

- Multi-factor verification with facial recognition and liveness detection to prevent spoofing.
- Optional GPS-based discovery for suggesting nearby parents.
- Community groups based on location or shared interests.
- Forums for advice support

2.4 Usage Scenario

Usage scenarios show how PlayDu helps parents form secure and meaningful connections from start to finish.

2.4.1 Example Scenario Walkthrough

Scenario: A New Parent Seeking Compatible Connections

1. A new parent downloads PlayDu and registers. They enter their parenting style such as Montessori, list personal interests like hiking and reading, and select life stage tags such as new parent. They upload their live selfie for third-party verification and receive a Verified badge once approved.
2. The parent browses suggested profiles generated. They like a profile that aligns with their parenting values. When the other parent also likes back, a match notification appears.
3. The two parents begin chatting within the secure in-app messaging system. They discuss parenting approaches and exchange ideas without sharing personal phone numbers.
4. The parent adjusts privacy settings, reviews safety guidelines, and can report or block users if needed.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By

simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.2 Use Case Level 0 (System Overview)

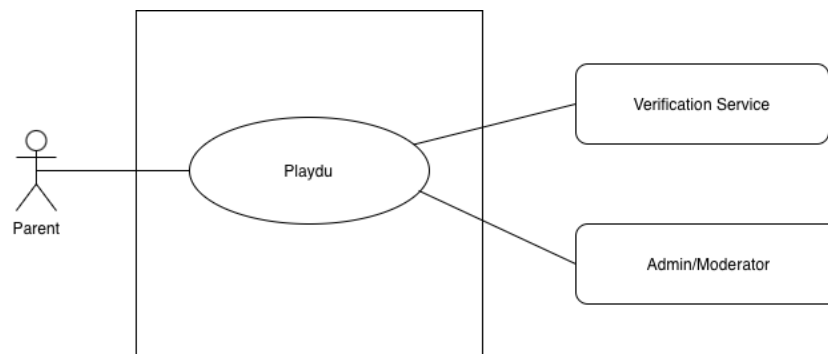


Fig: Use case level 0 (System)

Primary Actor: Parent (user of the PlayDu application)

Secondary Actor: Third-Party Verification Service (handles identity and ID verification), Admin/Moderator

Goal in Context: The diagram represents the high-level functionalities of PlayDu, a secure mobile platform for parents to build trusted communities for social, educational, and cultural engagement.

3.3 Use Case Level 1 (Detailed Modules)

This expanded diagram breaks down the main modules into specific functionalities:

Module 1: Profile & Verification Module

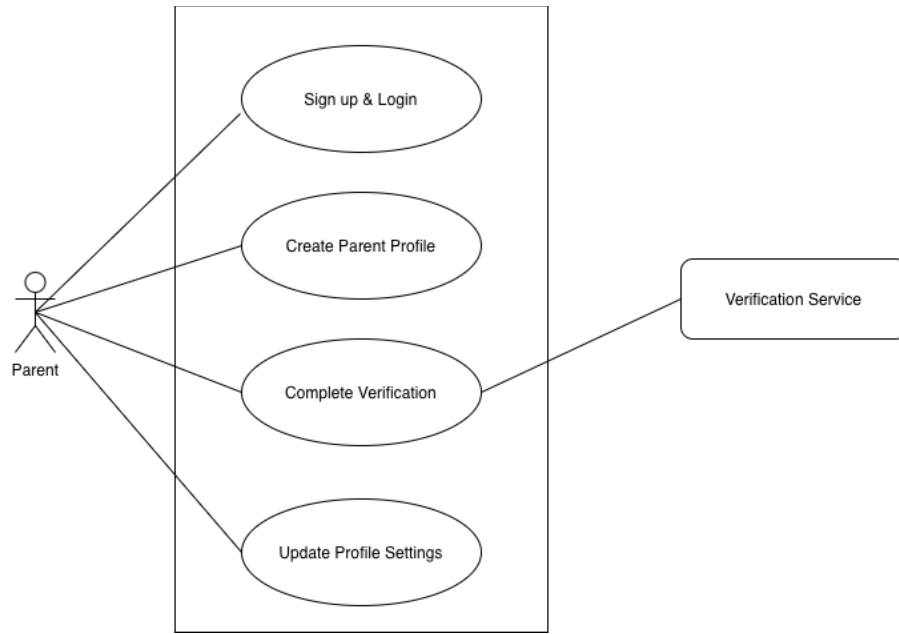


Fig: Use case level 1.1 (Profile & verification module)

Primary Actor: Parent (user of the PlayDu application)

Secondary Actor: Third-Party Verification Service (handles identity and ID verification)

Use Cases:

- **UC1:** Sign Up & Login - Parents create accounts with email/password and authenticate via JWT
- **UC2:** Create Parent Profile - Add parenting style, interests, and life stage tags
- **UC3:** Complete Verification - Upload government ID and pass third-party verification for "Verified" badge
- **UC4:** Update Profile Settings - Modify profile information and preferences

Module 2: Matching & Discovery Module

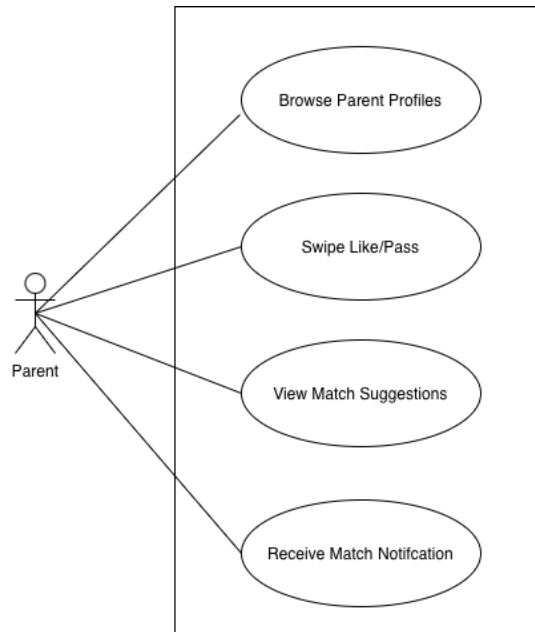


Fig: Use case level 1.2 (Matching & discovery module)

Primary Actor: Parent (user of the PlayDu application)

Use Cases:

- **UC5:** Browse Parent Profiles - View other verified parent profiles
- **UC6:** Swipe Like/Pass - Use swipe interface to express interest or pass on profiles
- **UC7:** View Match Suggestions - Receive AI-generated match recommendations
- **UC8:** Receive Match Notification - Get notified when mutual interest creates a match

ML Engine Integration: Uses hybrid recommendation system combining content-based filtering (shared interests, parenting philosophies) and collaborative filtering (user behavior patterns)

Module 3: Communication Module

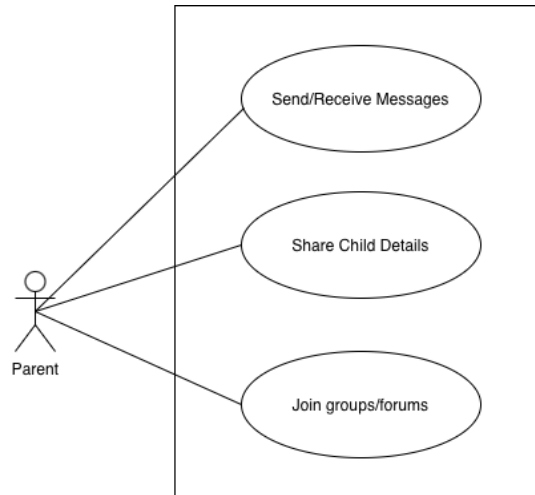


Fig: Use case level 1.3 (Communication module)

Primary Actor: Parent (user of the PlayDu application)

Use Cases:

- **UC9:** Send/Receive Messages - Secure in-app chat with matched parents
- **UC10:** Share Child Details - Voluntarily share private child information with matches
- **UC11:** Join groups/forums - Coordinate events and meetups

Module 4: Safety & Privacy Module

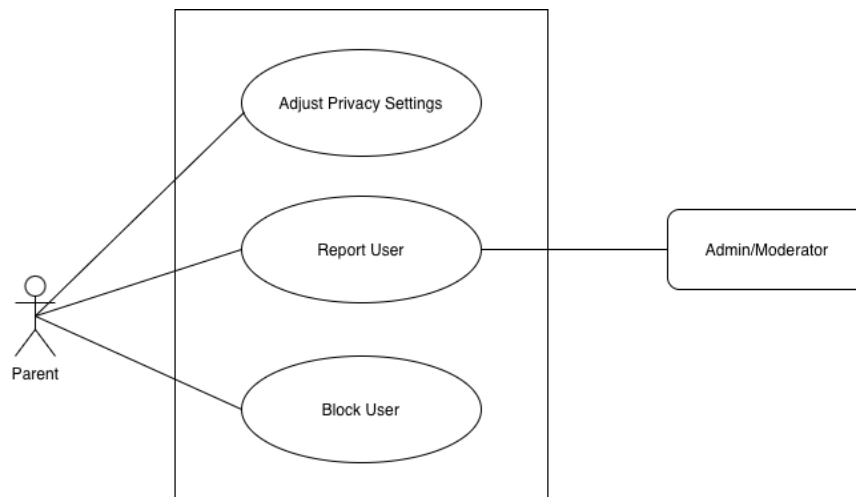


Fig: Use case level 1.4 (Safety & privacy module)

Primary Actor: Parent (user of the PlayDu application)

Secondary Actor: Admin/Moderator

Use Cases:

- **UC12:** Adjust Privacy Settings - Control profile visibility and information sharing
- **UC13:** Report User - Flag inappropriate behavior or profiles
- **UC14:** Block User - Prevent specific users from viewing profile or making contact

4. Data Based Modeling

Data based modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation. The entities identified for Playdu are:

- ☐ User - Core parent user accounts
- ☐ ParentingStyle - Categorization of parenting approaches
- ☐ LifeStage - Parent's current life situation
- ☐ Verification - ID and identity verification details
- ☐ Swipe - Record of like/pass actions
- ☐ Match - Mutual connections between parents
- ☐ Message - In-app communication
- ☐ Report - User-reported issues
- ☐ Block - Blocked user relationships
- ☐ Notification - System and user notifications

ER Diagram:

Attribute Name	Data Type	Constraints
user_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
full_name	VARCHAR(255)	NOT NULL
phone_number	VARCHAR(20)	NULL
bio	TEXT	NULL
profile_photo_url	VARCHAR(500)	NULL
account_status	ENUM('active', 'suspended', 'deactivated', 'pending_verification')	DEFAULT 'pending_verification'
is_verified	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
last_active	TIMESTAMP	NULL
privacy_settings	JSON	NULL
child_profiles	JSON	NULL (array of child profile objects)

VerificationTable:

verification_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE, UNIQUE

id_document_url	VARCHAR(500)	NOT NULL
face_photo_url	VARCHAR(500)	NULL
liveness_check_passed	BOOLEAN	DEFAULT FALSE
verification_provider	VARCHAR(100)	NULL
verification_status	ENUM('pending', 'approved', 'rejected', 'expired')	DEFAULT 'pending'

Swipe Table:		
Attribute Name	Data Type	Constraints
swipe_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
swiper_user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
swiped_user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
action_type	ENUM('like', 'pass')	NOT NULL
swiped_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Match table:		
match_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
user1_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
user2_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
matched_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

compatibility_score	DECIMAL(5,2)	NULL (0.00 to 100.00)
---------------------	--------------	-----------------------

Message Table

Attribute Name	Data Type	Constraints
message_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
match_id (FK)	INT	FOREIGN KEY → Match(match_id), ON DELETE CASCADE
sender_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
receiver_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
message_content	TEXT	NOT NULL
is_read	BOOLEAN	DEFAULT FALSE
sent_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
read_at	TIMESTAMP	NULL

Report Table:

Attribute Name	Data Type	Constraints
report_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
reporter_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE

reported_user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
report_type	ENUM('inappropriate_behavior', 'fake_profile', 'harassment', 'spam', 'other')	NOT NULL
report_description	TEXT	NOT NULL
status	ENUM('pending', 'investigating', 'resolved', 'dismissed')	DEFAULT 'pending'
reported_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
resolved_at	TIMESTAMP	NULL
admin_notes	TEXT	NULL

Block Table:

Attribute Name	Data Type	Constraints
block_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
blocker_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
blocked_user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
blocked_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Notification table:

Attribute Name	Data Type	Constraints
----------------	-----------	-------------

notification_id (PK)	INT	PRIMARY KEY, AUTO_INCREMENT
user_id (FK)	INT	FOREIGN KEY → User(user_id), ON DELETE CASCADE
notification_type	ENUM('new_match', 'new_message', 'group_invitation', 'group_post', 'verification_status', 'system_alert')	NOT NULL
notification_content	TEXT	NOT NULL
is_read	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
read_at	TIMESTAMP	NULL

5. Class Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the PlayDu project description, potential classes were filtered using General Classification(External entities, Things, Events, Roles, Organizational units, Places, and Structures) and Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements).

Initial Noun Identification

Based on PlayDu's description, the key nouns include:

1. Parent
2. User Profile
3. Child Profile
4. Parenting Style
5. Interest
6. Life Stage
7. Verification
8. Face Detection
9. Liveness Check
10. ID Document
11. Match
12. Swipe Action
13. Message
14. Chat
15. Event
16. Playdate
17. Report
18. Block
19. Safety Guidelines
20. Privacy Settings
21. Notification
22. Recommendation Engine
23. Filter
24. Community
25. Group
26. Forum
27. Discussion

Analysis and Filtering

Using General Classification and Selection Criteria, classes with similar responsibilities were identified and merged to avoid redundancy.

Derived Classes and Final Grouping

After analysis, the following core classes were identified:

1. User
2. VerificationService
3. Recommendation
4. MatchingService
5. Message
6. Group
7. Report
8. BlockedUser
9. Notification

5.2 Class cards

CRC Card (User):

User	
Attributes	Methods
- userId - email - passwordHash - fullName - phoneNumber - accountStatus - isVerified - createdAt - lastActive - parentingStyleId - lifeStageId - bio - profilePhotoUrl - interests[] - childProfiles[] - privacySettings - groupMemberships[]	- createAccount() - login() - logout() + updateProfile(data) + uploadIdDocument(document) + requestVerification() - changePassword(newPassword) + addInterest(interest) + removeInterest(interest) + setParentingStyle(styleId) + setLifeStage(stageId) + uploadPhoto(photo) + adjustPrivacySettings(settings) + blockUser(userId) + reportUser(userId, reason) + joinGroup(groupId) + leaveGroup(groupId)

	- calculateCompatibility(otherUser)
Responsibilities	Collaborators
• Manage user authentication and sessions • Store and handle complete user profile data • Manage parenting style and life stage • Handle interests and hobbies • Control privacy and safety settings • Interface with verification and matching services • Participate in community groups • Provide data for matching algorithms	• VerificationService • MatchingService • Recommendation • Message • Group • Report • BlockedUser • Notification

CRC Card (VerificationService):

VerificationService	
Attributes	Methods
- verificationId - userId - idDocumentUrl - facePhotoUrl - livenessCheckPassed - verificationProvider - verificationStatus - verifiedAt - expiresAt	+ initiateVerification(userId) + uploadIdDocument(document) + performFaceDetection(photo) + performLivenessCheck() + submitToThirdParty(data) - processVerificationResult(result) + approveVerification() + rejectVerification(reason) + checkVerificationStatus() + updateVerificationBadge()

Responsibilities	Collaborators
• Handle multi-factor ID verification process • Integrate with third-party verification services • Perform face detection and liveness checks • Manage verification status and badges • Store verification records securely	• User • Notification

CRC Card (Recommendation):

Recommendation	
Attributes	Methods
- filterCriteria - scoringWeights - collaborativeFilteringData - contentBasedData	+ generateRecommendations(userId) + applyFilters(criteria) - performContentBasedFiltering(profile) - performCollaborativeFiltering(userId) + detectMutualInterest(userId, targetId) + createMatch(user1Id, user2Id) - updateMLModel(feedback) + analyzeUserPreferences(userId) - weightProfileAttributes(user1, user2) + refreshRecommendations(userId)
Responsibilities	Collaborators

• Generate AI-powered match recommendations • Implement hybrid recommendation system • Calculate compatibility scores • Process swipe actions (like/pass) • Detect mutual interest and create matches • Continuously improve using ML feedback • Analyze user behavioral patterns • Weight and prioritize matching attributes	• User • MatchingService
--	---

CRC Card (MatchingService):

MatchingService	
Attributes	Methods
- matchId - user1Id - user2Id - matchedAt - compatibilityScore - conversationStarted - lastInteractionAt	+ createMatch(user1Id, user2Id) + getCompatibilityScore() + enableMessaging() + getConversationHistory() + allowChildInfoSharing()
Responsibilities	Collaborators

• Represent mutual connections between parents • Store match metadata and timestamps • Enable secure communication between matched users • Provide compatibility information • Facilitate child profile sharing after match	• User • Message • Group • Notification • Recommendation
---	--

CRC Card (Message):

Message	
Attributes	Methods
- messageId - matchId - senderId - receiverId - messageContent - isRead - sentAt - readAt - isEncrypted - messageType	+ sendMessage(matchId, content) + receiveMessage() + markAsRead() + getMessageContent() + deleteMessage() - encryptContent(content) - decryptContent(content) + validateMatchExists() + searchMessages(matchId, query) + deleteConversation(matchId)

Responsibilities	Collaborators
• Enable secure text communication between matches • Encrypt messages for privacy • Track message read/unread status • Ensure messages only sent within valid matches • Support real-time messaging • Provide message search functionality	• MatchingService • User • Notification

CRC Card (Group):

Group	
Attributes	Methods
- groupId - groupName - groupDescription - createdBy - createdAt - members[] - moderators[] - memberCount - posts[] - groupRules	+ createGroup(data) + addMember(userId) + removeMember(userId) + addModerator(userId) + removeModerator(userId) + postContent(userId, content) + deletePost(postId) + moderateContent(postId, action) + getGroupMembers() + getGroupPosts() + setGroupRules(rules) + deleteGroup()
Responsibilities	Collaborators
• Manage interest-based groups • Facilitate community discussions and forums • Track group membership and participation • Enable group discovery • Enforce group rules and guidelines • Provide space for advice sharing	• User • Matchingservice • Notification

CRC Card (Report):

Report	
Attributes	Methods
- reportId - reporterId - reportedUserId - reportDescription - reportedAt - resolvedAt - adminNotes - priority - evidenceUrls[] - actionTaken	+ fileReport(reporterId, reportedUserId, type, description) + assignToAdmin(adminId) + setPriority(level) + addAdminNotes(notes) + addEvidence(urls) + resolveReport(resolution) + dismissReport(reason) + notifyReporter(update) + getReportsByUser(userId) + takeAction(action)
Responsibilities	Collaborators
• Handle user-reported safety concerns • Categorize reports by type (harassment, fake profile, etc.) • Track report status and resolution • Facilitate admin review and action • Store evidence for investigation • Track actions taken on reports	• User • Notification • BlockedUser • Group

CRC Card (BlockedUser):

BlockedUser	
Attributes	Methods
- blockId - blockerId - blockedUserId - blockedAt - blockReason - isActive - canUnblock	+ createBlock(blockerId, blockedUserId) + unblockUser(userId) + isUserBlocked(blockerId, blockedUserId) + getBlockedUsers(userId) + getBlockReason() + setBlockReason(reason) + preventInteraction(user1, user2) + hideFromDiscovery(blockerId, blockedUserId)
Responsibilities	Collaborators
• Manage user blocking relationships • Prevent all interactions between blocked users • Track blocking history and reasons • Support unblocking functionality • Hide blocked users from discovery • Maintain blocking records	• User • Recommendation • Message • MatchingService • Report • Group

CRC Card (Notification):

Notification	
Attributes	Methods
- notificationId - userId - notificationType - notificationContent - isRead - metadata - createdAt - readAt - priority	+ sendNotification(userId, type, content) + markAsRead() + markAllAsRead(userId) + deleteNotification() + getNotificationHistory(userId) + updateNotificationPreferences(prefs) - determinePriority(type)
Responsibilities	Collaborators
• Deliver real-time notifications to users • Manage notification types (match, message, group, etc.) • Track read/unread notification status • Respect user notification preferences	• User • Match • Message • Group • VerificationService • Report

5.3 CRC Diagram

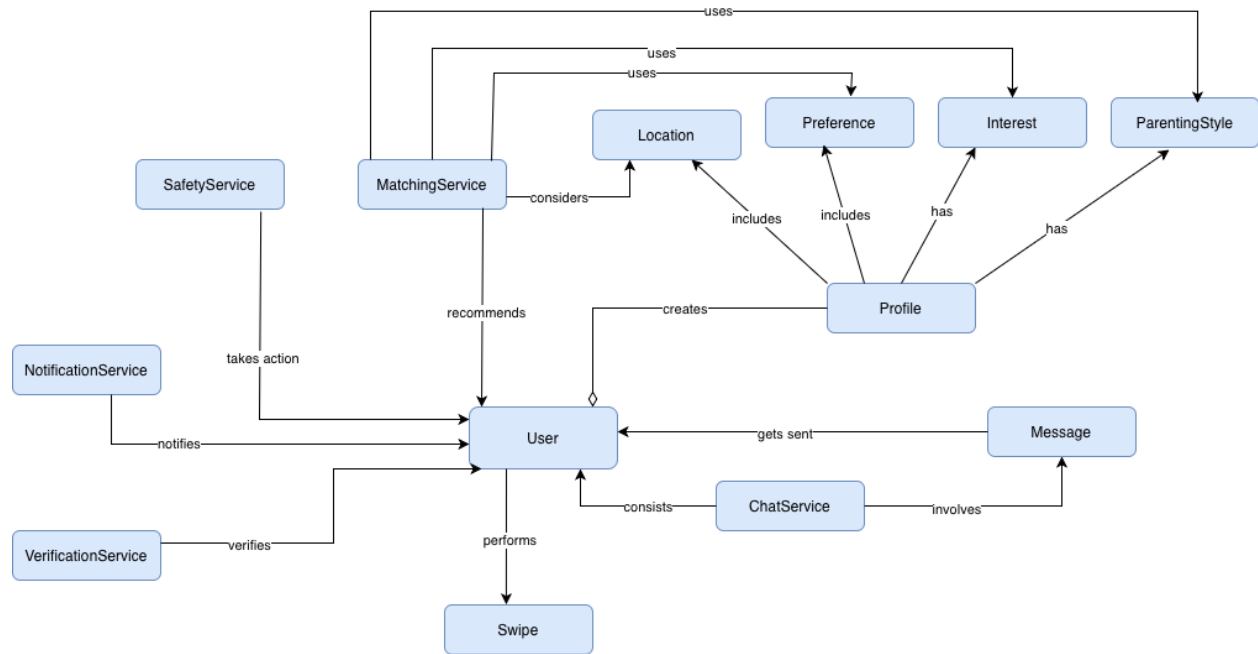


Fig: CRC diagram

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

The architectural context diagram for the is given below:

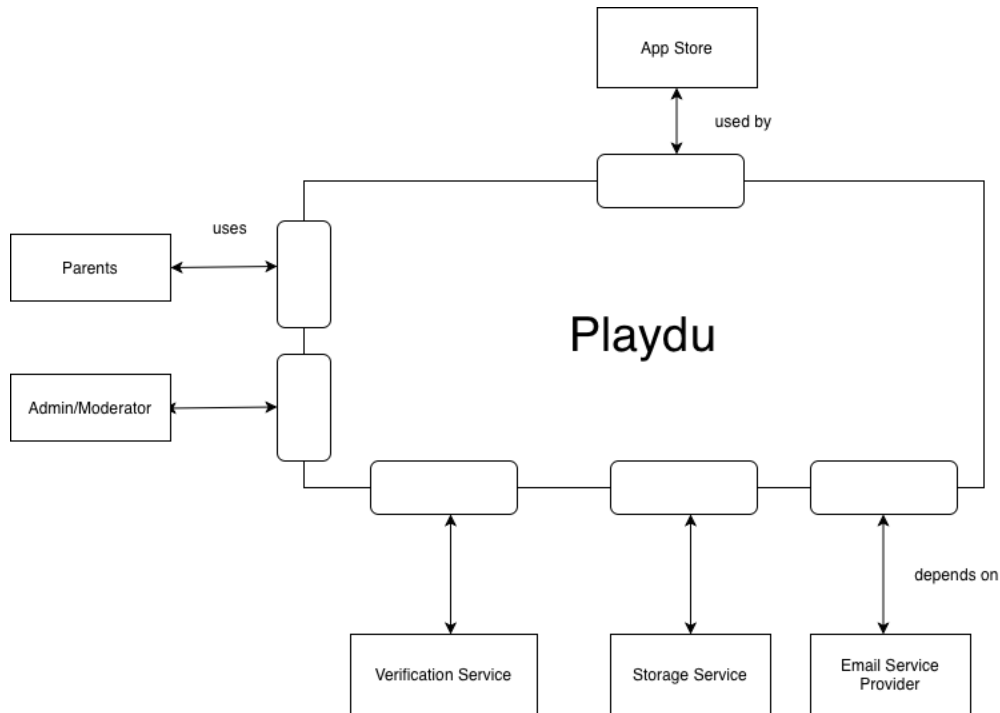


Fig: Architectural Context Diagram

Users: Parents, Admin/Moderator

Superordinate Systems: App Store/Play Store

Subordinate Systems: Verification Service, Storage Service, Email Service

6.2 Archetypes

The system follows a four-layer architectural pattern with clear separation of concerns. This layered approach ensures modularity, maintainability, security, and scalability. Each layer has specific responsibilities and interacts with adjacent layers through well-defined interfaces.

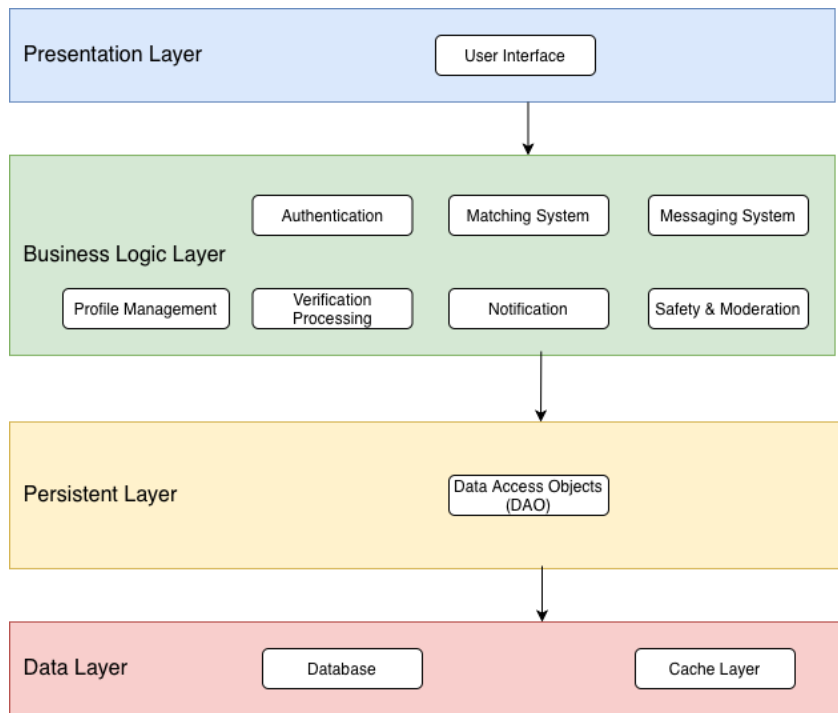


Fig: Archetype of Playdu

1. Presentation Layer

The Presentation Layer is the user-facing component that handles all interactions with parents using the mobile application.

Components:

User Interface (React Native + Expo)

- Renders all mobile screens and visual elements
- Implements the swipe-based matching interface
- Displays chat conversations and match lists
- Manages settings, privacy controls, and safety features
- Supports responsive design for various screen sizes
- Handles gestures (swipe, tap, scroll)
- Displays real-time notifications

Key Screens Include:

- Registration/Login
- Profile Creation & Editing
- Swipe/Discovery Interface
- Match List & Chat
- Settings & Privacy

- Safety & Reports

2. Business Logic Layer

The Business Logic Layer contains the core application intelligence and orchestrates all business operations.

Components:

Authentication (JWT-Based)

- Manages user login and signup workflows
- Implements password hashing
- Generates and stores JWT tokens securely
- Handles token refresh mechanisms
- Manages session persistence
- Enforces access control based on verification status
- Provides logout functionality

Profile Management

- Creates and updates parent profiles (name, bio, parenting style, interests)
- Handles profile photo uploads
- Validates profile completeness
- Controls profile visibility based on privacy settings

Matching Engine

- Generates AI-powered match recommendations
- Processes swipe actions (like/pass)
- Calculates compatibility scores between parents
- Detects mutual interest and creates matches
- Integrates with ML Integration Module for recommendations
- Updates recommendation algorithm based on user feedback

Messaging System

- Facilitates secure text communication between matched users
- Validates that messages only sent within active matches
- Manages conversation threads and history
- Tracks message read/unread status
- Integrates with Encryption Service for message security
- Supports real-time message delivery

Verification Processing

- Orchestrates multi-step ID verification workflow
- Processes uploaded ID documents
- Coordinates with Third-Party Verification API
- Performs face detection on profile photos
- Executes liveness checks
- Updates user verification status
- Awards "Verified" badges to authenticated users
- Handles verification failures with retry mechanisms

Notification Manager

- Delivers real-time notifications to users
- Manages notification types

- Respects user notification preferences
- Tracks notification read/unread status

3. Persistent Layer

The Persistent Layer provides specialized services that support the business logic layer with complex operations.

Components:

Data Access Objects (DAOs)

- Provides abstraction between business logic and database
- Executes CRUD operations (Create, Read, Update, Delete)
- Implements data validation and sanitization

4. Data Layer

The Data Layer manages all data persistence, retrieval, and caching operations.

Components:

Database

- Stores all persistent application data
- Maintains data integrity with ACID properties
- Provides backup and recovery mechanisms
- Stores: users, profiles, matches, messages, reports, verification records etc.

Cache Layer (Redis)

- Caches user sessions and JWT tokens
- Stores frequently accessed data (active matches, recent messages)
- Reduces database load for read-heavy operations
- Caches recommendation results temporarily

6.3 Top Level Components

Overall architectural structure with top level components is illustrated below:

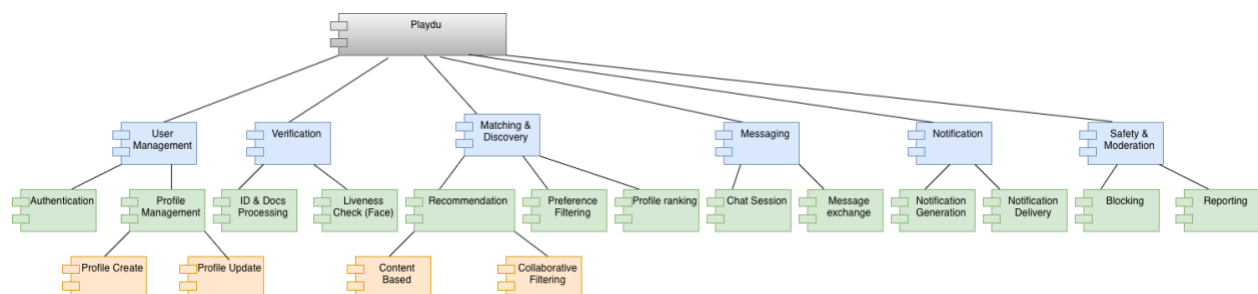


Fig: Top level components

6.4 Deployment Diagram

Deployment Diagram shows how components are physically deployed across client devices, web servers, backend containers, databases, and external services

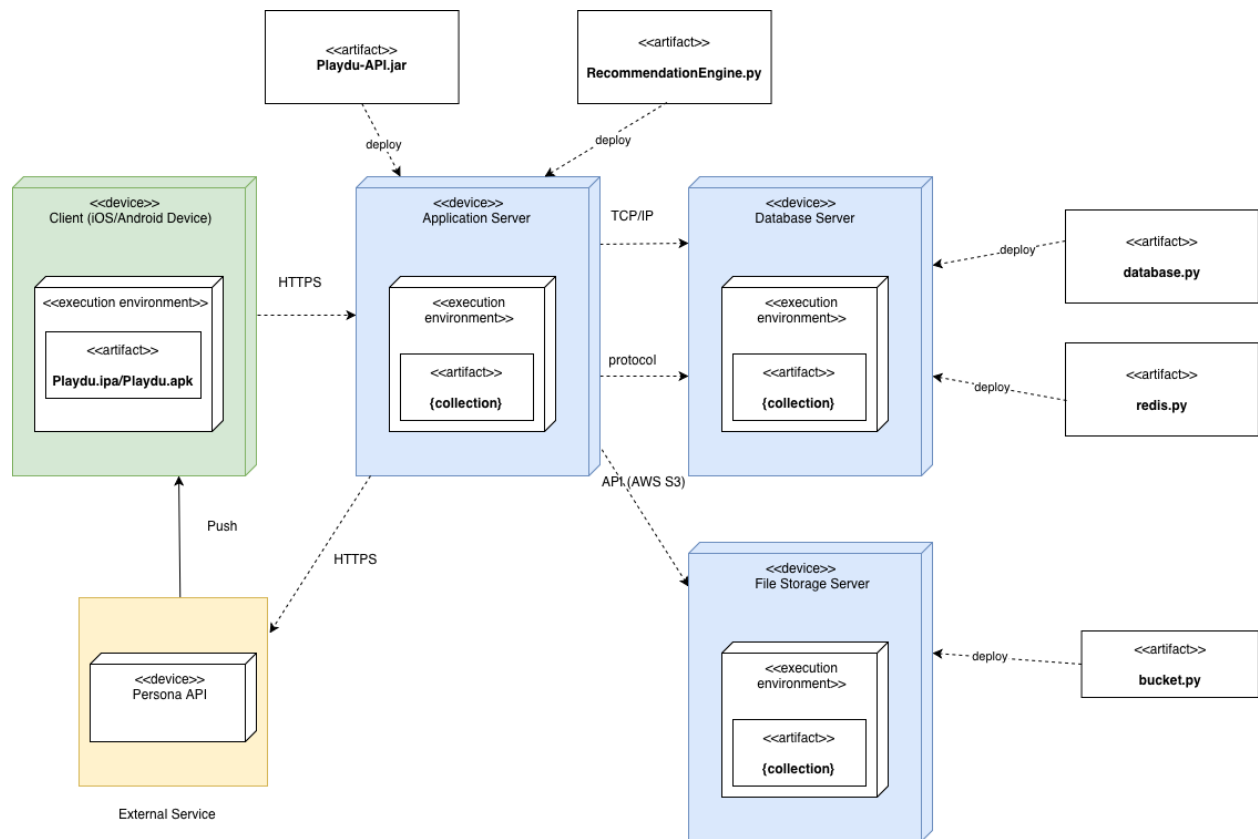


Fig: Deployment Diagram

This architecture supports PlayDu's core mission of providing secure, AI-powered parent connections while maintaining flexibility for future enhancements and scalability for growing user base.

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

The primary testing goals for the Playdu are as follows:

- Ensure that PlayDu meets all normal, expected, and exciting requirements as specified in the project description, including profile creation, verification, matching, messaging, and safety features.
- Test that the verification process (face detection, liveness checks, third-party ID verification) works correctly and prevents unauthorized access or fake profiles.
- Confirm that the hybrid recommendation system (content-based and collaborative filtering) generates relevant match suggestions based on shared interests, parenting styles, and values.
- Validate that in-app messaging is secure, and only available between matched users, with no unauthorized access to conversations.
- Test the application's responsiveness under various loads, ensuring profile loading, matching, and messaging respond within acceptable time limits.
- Verify that the user interface is intuitive, responsive across mobile devices, and supports basic accessibility features.
- Ensure that all data storage, retrieval, and updates maintain consistency and accuracy across the system.
- Confirm that the system handles edge cases and errors gracefully, providing meaningful feedback to users.

7.2 Test Cases

Below is a comprehensive set of test cases designed to evaluate PlayDu's functionality, security, performance, and usability.

Test Case 1: Google Sign-In Registration and Login

Test Scenario: A parent registers and logs in via Google OAuth.

Prerequisites: Valid Google account

Steps:

1. Tap "Continue with Google" and complete Google OAuth
2. If email is new, complete profile setup. If email exists, observe auto-login.

Expected Outcomes:

- New Google users skip name/password entry. Full name is taken from the Google token.
- Existing Google users are auto-logged in silently with no extra steps

Result: Passed

Test Case 2: Parent Profile Creation

Test Scenario: A logged-in user completes their parent profile.

Prerequisites: User registered and logged in

Steps:

1. Upload a profile photo on the Upload Photo screen
2. Fill in name, parenting style, life stage tags, interests, bio, and location on the Profile Setup screen
3. Submit the profile

Expected Outcomes:

- All fields are saved and returned correctly by GET /profile/me
- Profile photo is stored in MinIO; photo_filename is updated
- profile_completed is set to true after a valid submission

Result: Passed

Test Case 3: Face Verification

Test Scenario: User completes a face liveness check to confirm their identity matches their profile photo.

Prerequisites: Profile photo uploaded

Steps:

1. Navigate to the Liveness screen and submit a real-time selfie
2. Submit a very low-resolution or pitch-black image
3. Submit a selfie that clearly does not match the profile photo

Expected Outcomes:

- A valid selfie matching the profile photo (Face++ confidence ≥ 75) sets `liveness_verified`: true
- Unreadable images (too small or too dark) are rejected before the API call
- A mismatched face returns a failure message; `liveness_verified` remains false

Result: Passed

Test Case 4: Match Recommendation Generation

Test Scenario: User receives profile suggestions based on compatibility and swipe history.

Prerequisites: Profile completed, other user profiles exist in the database

Steps:

1. Navigate to the Discover screen and observe the suggestions
2. Swipe on several profiles, then reload suggestions
3. Verify a blocked user does not appear in suggestions

Expected Outcomes:

- Suggestions exclude the current user, and blocked users
- Profiles are ranked by a hybrid score of shared interests, parenting style, and other attributes
- The discovery feed updates correctly after each swipe action

Result: Passed

Test Case 5: Swipe Actions (Like/Pass)

Test Scenario: User likes and passes profiles from the discovery feed.

Prerequisites: Suggestions are available on the Discover screen

Steps:

1. Swipe right (Like) on a profile
2. Swipe left (Pass) on a profile
3. Reload the discovery feed

Expected Outcomes:

- Both swipe directions are correctly recorded via POST /matching/swipe
- Passed profiles go to the bottom pool, liked profiles feed into collaborative filtering
- If the liked user had already liked back, the response immediately returns matched: true

Result: Passed

Test Case 6: Mutual Match Creation

Test Scenario: Two users like each other, creating a match.

Prerequisites: Both users have completed profiles and are visible to each other

Steps:

1. User A swipes right on User B
2. User B swipes right on User A

Expected Outcomes:

- User B's swipe response returns { matched: true, match_id: <id> }
- A Match row is created in the database
- A "match" notification is created for both users
- The match appears in both users' matches list and messaging is enabled

Result: Passed

Test Case 7: In-App Messaging

Test Scenario: Matched users exchange messages in real time.

Prerequisites: Users A and B have an active match

Steps:

1. User A opens the chat with User B and sends a message
2. User B opens the conversation and replies
3. Attempt to send a message to a non-matched user

Expected Outcomes:

- Messages are delivered in real time via WebSocket
- Read receipts update when the recipient views the message
- Sending to a non-matched user returns HTTP 403/404
- Edited messages are supported; timestamps are stored and displayed

Result: Passed

Test Case 8: User Reporting

Test Scenario: User reports another user for inappropriate behavior.

Prerequisites: User is authenticated

Steps:

1. Submit a report against another user with a valid report_type and description
2. Attempt to submit a report with an empty report_type

Expected Outcomes:

- Valid reports are stored with reporter ID, target ID, type, description, and timestamp (HTTP 200)
- Empty report_type returns HTTP 400

Result: Passed

Test Case 9: User Blocking

Test Scenario: User blocks another user to prevent further contact.

Prerequisites: User is authenticated

Steps:

1. Block another user via POST /safety/block
2. Check the discovery feed and matches list
3. Unblock the user

Expected Outcomes:

- Blocked user disappears from discovery suggestions and match interactions
- Unblocking re-enables access
- Blocked user receives no notification of being blocked

Result: Passed

Test Case 10: Matching Algorithm Accuracy

Test Scenario: Recommendations reflect profile compatibility and swipe history.

Prerequisites: Diverse user profiles in the database

Steps:

1. Create a test user with specific interests ("Hiking", "Montessori", "Book Clubs") and a set location
2. Fetch suggestions and inspect the top results
3. Simulate several swipes and re-fetch suggestions

Expected Outcomes:

- Top recommendations share at least 1 interest or the same parenting style with the test user

- Closer users rank higher when a distance preference is set
- Collaborative filtering shifts recommendations after swipes, already-swiped users are excluded

Result: Passed

Test Case 11: Notifications

Test Scenario: Notification system creates, delivers, and tracks notifications correctly.

Prerequisites: Two users with an active match

Steps:

1. User C likes User A (no mutual match yet), check User A's notifications
2. User C and User A mutually match, check both notification lists
3. Mark a notification as read, delete another

Expected Outcomes:

- Match events produce Notification rows for the relevant user(s)
- A one-sided like does not create a notification (privacy)
- POST /notifications/<id>/read sets is_read: true with a timestamp, DELETE removes it

Result: Passed

Test Case 12: Rate Limiting

Test Scenario: Auth endpoints enforce per-IP request limits.

Prerequisites: None

Steps:

1. Send 6 rapid POST requests to /auth/register from the same IP
2. Send 11 rapid POST requests to /auth/login from the same IP

Expected Outcomes:

- The 6th register request returns HTTP 429 (limit: 5/minute)
- The 11th login request returns HTTP 429 (limit: 10/minute)
- Requests within the limit complete with expected status codes

Result: Passed

Test Case 13: Session Management and Logout

Test Scenario: JWT-based session persistence, expiry, and manual logout behave correctly.

Prerequisites: User is logged in

Steps:

1. Close and reopen the app within the JWT validity window
2. Attempt an API call after the JWT has expired
3. Log out manually from profile settings

Expected Outcomes:

- JWT is stored securely
- An expired token causes any protected request to return HTTP 401 and redirects to the login screen
- Manual logout clears the stored token, protected routes are inaccessible until re-login

Result: Passed

Test Case 14: Data Integrity and Validation

Test Scenario: Ensure data consistency across system operations.

Prerequisites: User has active profile, matches, and messages

Steps:

1. User updates profile information
2. Verify blocked user data is preserved but access is restricted

Expected Outcome:

- Profile updates propagate to all related data

- Blocking enforces access restrictions without data loss

Result: Passed

Test Case 15: Forum Feed and Tag Filtering

Test Scenario: User browses the community feed and filters posts by topic.

Prerequisites: User is authenticated with a completed profile, some posts exist in the database

Steps:

1. Navigate to the Forum (Community) tab and observe the default feed
2. Tap a topic tag (e.g., "Playdates") in the horizontal tag bar
3. Pull down to refresh

Expected Outcomes:

1. Feed loads posts ranked by high-engagement.
2. Tag filter correctly narrows results to the selected tag only, "All" shows every post
3. Posts from blocked users are excluded from the feed regardless of filter

Result: Passed

Test Case 16: Creating a Forum Post

Test Scenario: User creates a new post with a title, body, topic tag, and optional image.

Prerequisites: User is authenticated

Steps:

1. Tap the + Post button on the Forum screen
2. Enter a title and body, select a topic tag, and submit
3. Attempt to submit without a title or tag

Expected Outcomes:

- Valid post appears in the feed immediately after submission with the correct tag pill, author name, and timestamp
- Image uploads correctly
- Missing title or tag is rejected with an appropriate validation error

Result: Passed

Test Case 17: Post Reactions (Upvote / Downvote)

Test Scenario: User upvotes and downvotes posts from the feed.

Prerequisites: Posts are visible in the feed

Steps:

1. Tap the upvote button on a post card
2. Tap again on the same post
3. Tap the downvote button on a different post
4. Tap again on the same post

Expected Outcomes:

- First tap records the reaction and increments the count immediately in the UI
- Tapping the same reaction again toggles it off (undo), count decrements
- A user cannot hold both upvote and downvote on the same post simultaneously, switching replaces the previous reaction

Result: Passed

Test Case 18: Comments and Nested Replies

Test Scenario: User adds a comment to a post and replies to an existing comment.

Prerequisites: Posts are visible in the feed

Steps:

1. Type a comment in the input box and tap Send
2. Tap Reply on an existing comment and submit a reply
3. Attempt to reply to a nested reply
4. Delete own comment

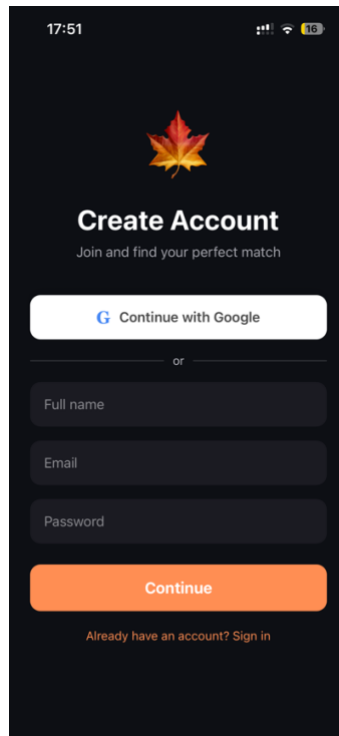
Expected Outcomes:

- Comment appears immediately, comment count on the post card increments
- Reply is nested visually under the parent comment (one level deep)
- Second-level nesting is not supported
- Deleting own comment removes it

Result: Passed

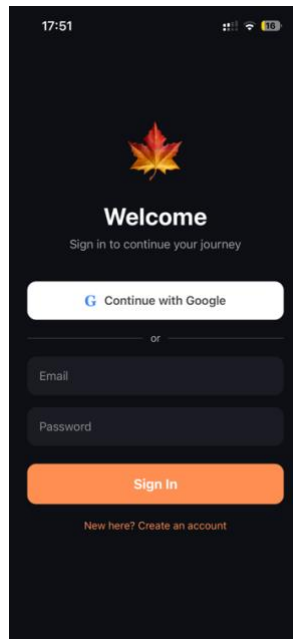
8. User Manual

Signup



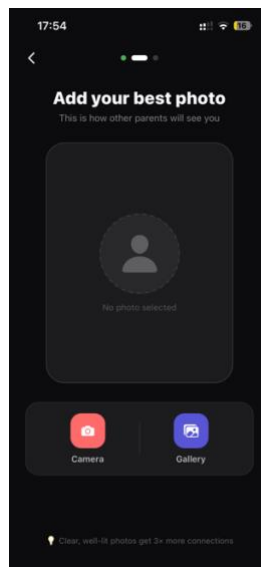
Tap Continue with Google to register without a password. The app uses your Google account's name and email automatically. No password entry is required.

Signin



Tap Continue with Google for Google-authenticated accounts.

Upload Photo Screen



1. Tap the camera/image area to open the image picker.
2. Select a photo from your library or take a new one.
3. Tap Continue to proceed to Profile Setup.
4. Tap the back arrow to return to Register.

Profile Setup

17:57

Set Up Profile Done

Help other parents get to know you

About You

Bio *

Share a bit about yourself — your values, what you love, your parenting vibe...

Date of Birth

Used to show your age to matches. Not displayed publicly.

Month Day Year

Location

Use My Current Location

— or search for a city —

e.g. London, Toronto, Sydney...

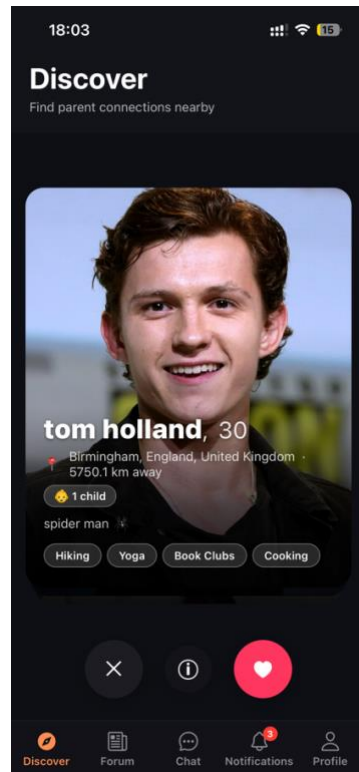
Discovery Distance

How far away should suggested parents be?

25 miles (40 km)

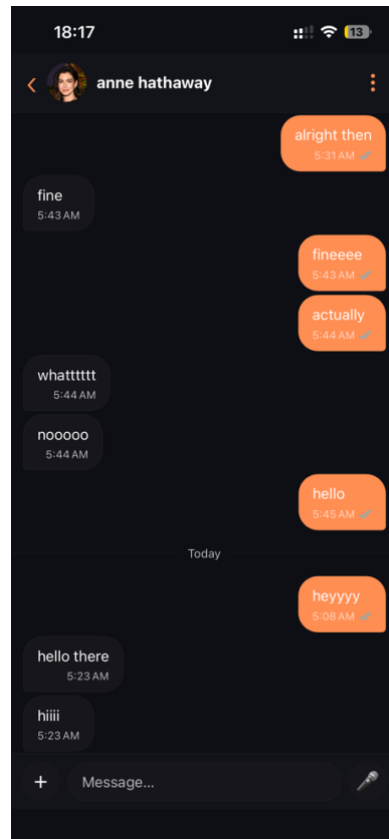
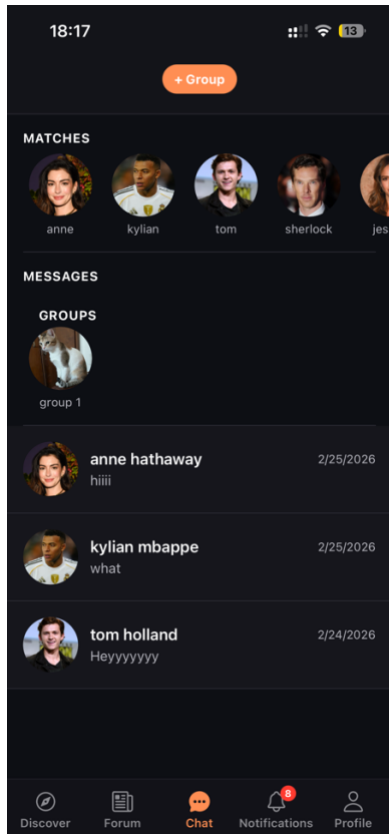
1. Fill all the mandatory info.
2. You can skip optional ones.
3. Tap “Done” to navigate to the main discover tab.

Discover



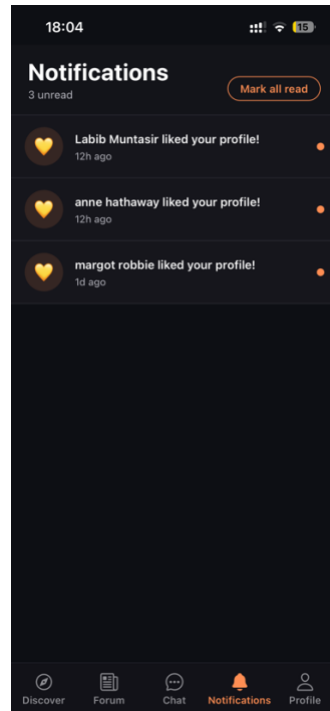
1. Cards are stacked and displayed one at a time. Each card shows the parent's photo, name, age, location, distance, parenting style, and shared interests.
2. Swipe right or tap the heart button to Like a profile.
3. Swipe left or tap the X button to Pass.
4. If a Like results in a mutual match, a match banner is shown immediately.
5. The deck refreshes automatically when all cards have been swiped.
6. Tapping the info button opens a full profile view.

Chat



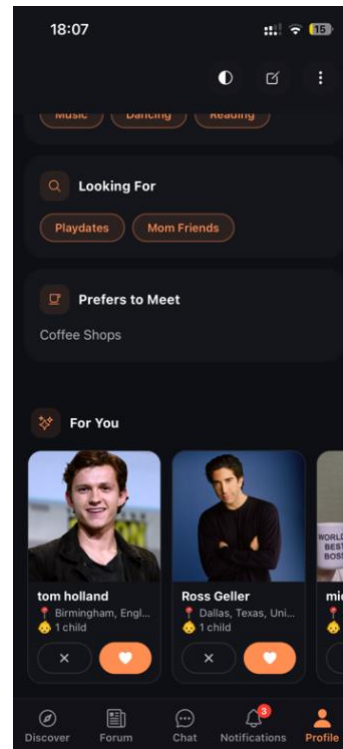
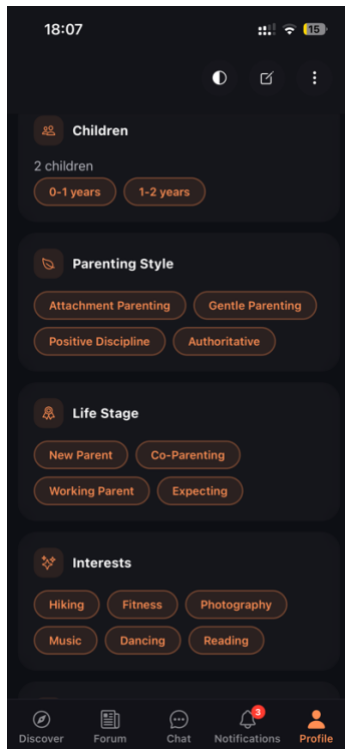
1. The Chat tab lists all your matches, each showing the last message preview, timestamp, and an unread message badge.
2. Tap a match to open the conversation.
3. Type in the text field at the bottom and tap Send.
4. A double-tick icon indicates a read message.
5. Long-press a message to edit or delete it.
6. Tap the mute icon to silence notifications for that conversation.

Notification



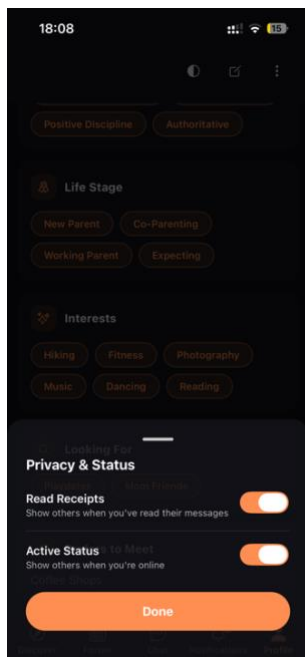
1. Notifications are listed newest-first.
2. Unread notifications are visually highlighted.
3. Tap a notification to navigate to the relevant content (match, profile, or post).
4. Tap the notification to mark it as read.

Profile (Own)



1. Profile card: displays your photo, name, age, location, bio, and all tags.
2. Extra Photos: up to several additional photos can be uploaded or removed.
3. For You: a mini-discovery feed showing suggested profiles directly on the Profile screen, allowing quick likes without going to the Discover tab.

Settings menu (: button):



1. Edit Profile: edit any profile field
2. Privacy Settings: toggle global read receipts and active status visibility
3. Theme: switch the app's colour scheme
4. Log Out: returns to the Login screen

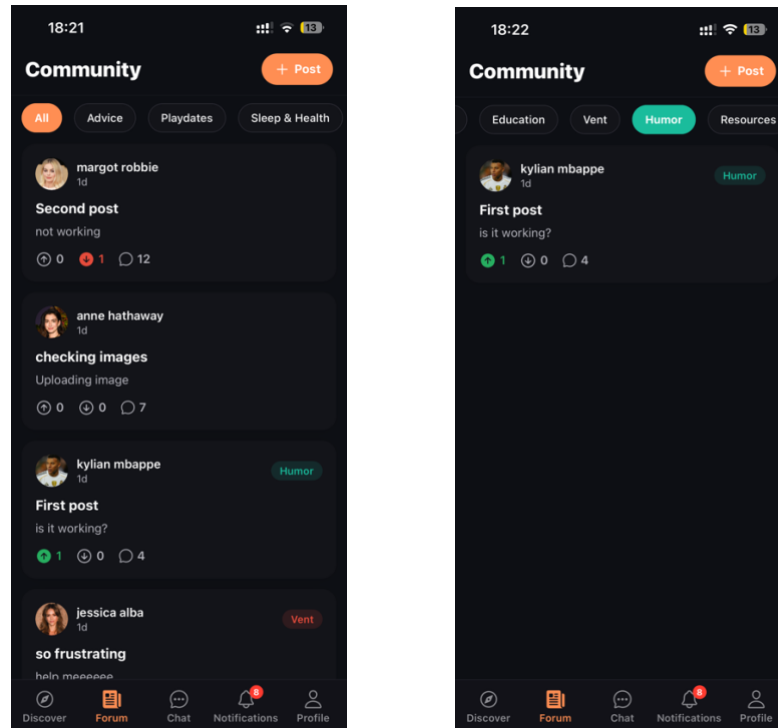
Profile (Other)

1. Reached by tapping a card on Discover, a match in Chat, or a notification.
2. Shows the user's photo gallery, bio, parenting style, life stage tags, interests, and "Looking For" list.
3. If the user has passed the liveness check, a Verified badge is displayed on their photo.
4. Tap the heart/X buttons to Like or Pass directly from the profile view.

Face Verification

1. Navigate to the Liveness screen from profile settings.
2. Follow the on-screen instructions and take a real-time selfie using the front camera.
3. Tap Submit.
4. The selfie is compared against your profile photo.
5. A pitch-black or too-small image is rejected before the API call. The verification badge appears on your profile card once passed.

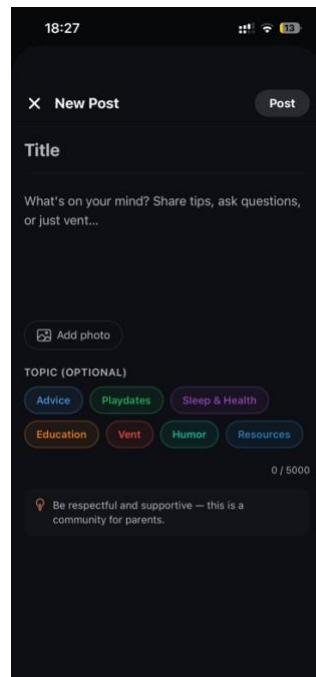
Forum



Viewing the Feed

1. The Forum tab opens a paginated feed of posts sorted by recent posts with high engagement and topic relevance to your profile appear first.
2. Each post card shows the author's photo and name, relative timestamp, topic tag pill (colour-coded), title, body preview, upvote/downvote counts, and comment count.
3. Tap the tag pills in the horizontal scroll bar at the top to filter the feed by topic: All, Advice, Playdates, Sleep & Health, Education, Vent, Humor, Resources
4. Pull down to refresh

Creating a Post

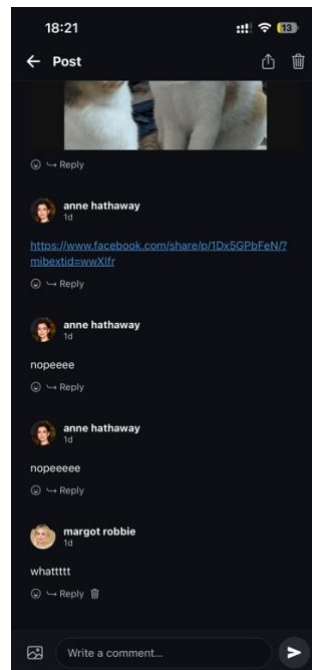


1. Tap the + Post button in the top-right corner.
2. Enter a title and body. Optionally attach an image.
3. Select a topic tag from the list (optional).
4. Tap Submit. The feed refreshes automatically on return.

Reacting to Posts

1. Tap the upvote or downvote arrows directly on a post card in the feed.
2. Tapping the same reaction again toggles it off (undo).

Comments and Replies



1. Tap a post card to open the full post view.
2. Type in the comment box at the bottom and tap Send to add a comment.
3. Tap Reply under any comment to add a nested reply.
4. You can delete your own comments.

9. Conclusion

This report explains the technical details behind PlayDu, a secure mobile app for parent connections. It describes the project's motivation, scope, requirements, and usage scenarios. These sections create a clear foundation for building the system. The report also explains the testing plan, including authentication, verification, matching, communication, privacy, performance, and accessibility. This ensures the app meets both functional and non-functional requirements.

The document includes diagrams such as use case diagrams, ER diagrams, CRC diagrams, and architecture illustrations. These visuals make the system design easier to understand. The modular structure and clear testing strategy help ensure the prototype can be completed within the semester while maintaining security and quality.

Overall, this report acts as a technical blueprint for supervisors and stakeholders. It provides a complete view of the system's design and development approach. PlayDu addresses a real social problem by helping parents overcome isolation through safe connections. From the initial idea to this midterm stage, the focus has remained on building a trusted digital space where parents can connect, share experiences, and support one another.